

Local Supermarket Management System for Small Shops

CST2550 Reset Coursework Report

Module	CST2550 Reset Coursework
Application Type	C# .NET 10 Windows Forms Application
Database	SQL Server LocalDB with Entity Framework Core
Student Name	Karish Shahi
Student ID	M01010883

1. Introduction and Supported Supermarket Operations

Small supermarkets usually use notebooks or simple excel sheets to manage their products, stock, suppliers, and sales. This type of manual process leads to problems like duplicate product records, delayed re-stocking and incorrect stock data. This Local Supermarket Management System will automate these functionalities, providing a solution for these problems.

The application is developed using C#.NET. Windows Forms was selected because it supports forms, buttons, input fields, data grids, and event-driven interaction, which is suitable for a small business application requiring functional data entry screens and transparent tables (Microsoft, 2025a). SQL Server LocalDB for local persistent storage Entity Framework Core maps C# entity classes to database tables and gets rid of repetitive data-access code (Microsoft, 2024).

The system can manage products, suppliers, stock, sales recording, search, monitoring of low stock and generation of simple reports. Each product record contains product ID, barcode, title, brand, category, supplier, price, quantity in stock, low stock threshold, stock status, expiry date and restock date. The fields are suitable for the supermarket operations outlined in the coursework brief and are adequate for managing the day-to-day running of the shop.

- Product & inventory management add, change, delete, search, monitor inventory level
- Supplier management: contacts, information and addresses of suppliers.
- Sales management – Add products to cart, do transactions and reduce stock automatically.
- Reporting: low stock products reporting, sales by product, products by category and supplier stock reporting

This system is purposefully designed in accordance to the needs of a small shop. It has all the basic core workflow a supermarket staff would need. Advanced features like customer accounts, receipt printing etc. are intended for future developments.

Problem Statement

The main problem is that paper based supermarket system does not give fast and accurate access to information. A product might exist but its information could be wrong. Similarly, sales reduces stock level, but if this is not properly recorded, it can cause problems. To address these issues, this system stores all that data into a single database. The records are updated based on the sales.

The system also provided utility functions. The low stock reports tell the users which product needs to be restocked while sales by product shows which product is most selling. These operations facilitate the workflow of the supermarket.

2. System Design and User Interface

The UI layer consists of Windows Forms Tabs and Controls. The service layer hosts the business logic for products, suppliers, stock, sales and reporting. The data layer consists of Entity Framework Core database context, model classes, and seed data. This is a good separation for the code as it makes it easier to maintain, because the interface changes, business logic and database access are not all mixed up in one large class.

The application is tabbed and has 4 sections namely

- Products and Stock
- Suppliers
- Sales
- Reports.

Product and Stock screen user can add a new product, set its stock limits, select category, search for products and update product data. The supplier section contains information regarding all suppliers. A cart workflow is implemented in the sales section to perform purchase transactions. The reports section provides useful summaries to help with decision making.

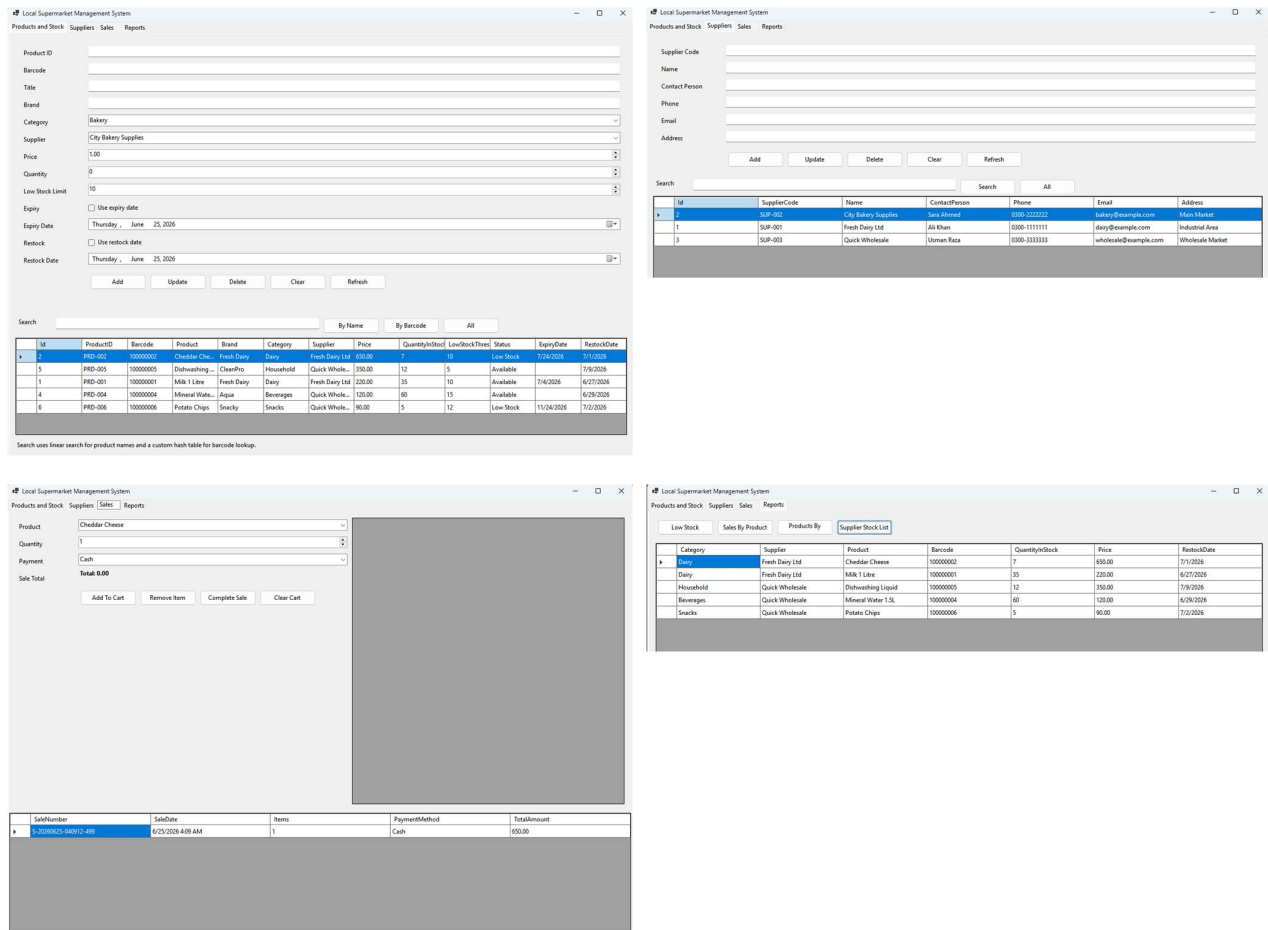


Figure 1: UI of the Application

The interface has intuitive labels, data grids to show records, action buttons and input forms. This type of software is ideal for non-technical users.

3. Database Design and Relationship Diagram

The database contains six tables:

- Categories
- Suppliers
- Products
- StockRecord
- Sales
- SaleItems.

The main table is the Products table because every product belongs to a category and to a supplier and can have many stock records and can be part of many sale items. Sales and SaleItems tables are used for transactions. Each sale consists of a number of items sold and this layered architecture helps reduce redundancy.

Table	Purpose
Categories	Stores product categories such as Dairy, Bakery, Beverages, Household and Snacks.
Suppliers	Stores supplier code, name, contact person, phone, email and address.
Products	Stores product details, price, stock quantity, thresholds and date fields.
StockRecords	Stores stock movement history such as additions, updates and sale reductions.
Sales	Stores sale number, date, payment method and total amount.
SaleItems	Stores the individual products and quantities included in each sale.

Microsoft SQL Server LocalDB was used as it provides a lightweight SQL Server database for development and testing locally (Microsoft, 2026). Entity Framework Core generates and modifies the database schema according to the model classes of the application. The service layer and the database configuration enforce validation rules such as unique product codes, unique barcodes, positive prices, valid quantities, required supplier details and required category details.

Local Supermarket Management System - Database Relationship Diagram

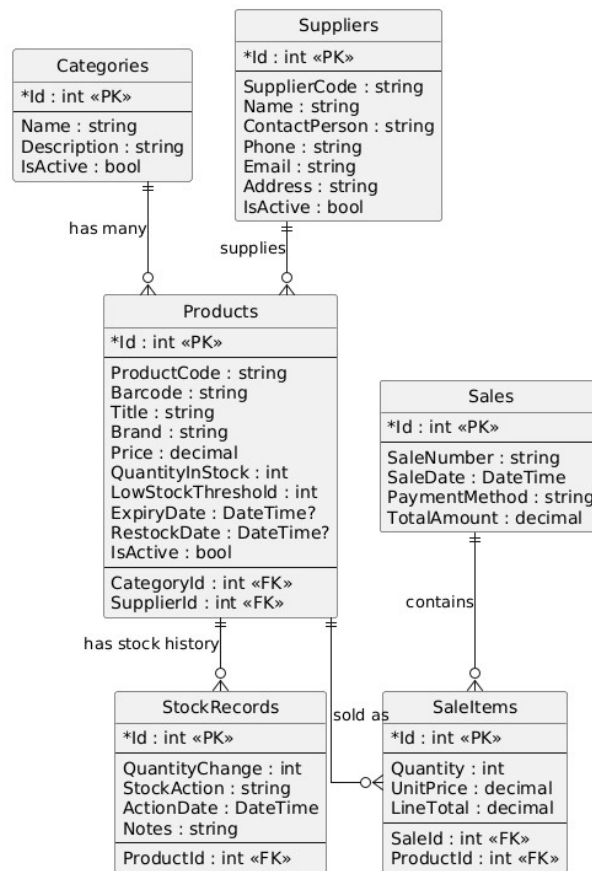


Figure 2: Database relationship diagram / ERD for the supermarket system

4. Data Structures, Algorithms and Time Complexity

The coursework requires custom data structures to manage and search supermarket information. The system therefore uses a custom linked list for sequential product traversal and a custom hash table for barcode lookup. The custom hash table is suitable for barcode searching because a barcode is a unique key and should retrieve one product quickly.

Two search algorithms are implemented. Product-name search uses linear search over the custom linked list. Barcode search uses a custom hash table and retrieves the product by key. This creates a useful

comparison between a flexible $O(n)$ search and an efficient $O(1)$ average-case lookup. Hash tables are widely used for fast key-based retrieval when collisions are managed effectively (Cormen et al., 2022).

Operation	Method / Structure	Complexity	Explanation
Add product	Validation + database insert	$O(n)$	Existing products may be checked for duplicate product code and barcode.
Search by product name	Custom linked list + linear search	$O(n)$	Each product title may need to be compared with the search text.
Search by barcode	Custom hash table	$O(1)$ average / $O(n)$ worst	Average lookup is fast; worst case occurs if many keys collide.
Update stock	Product lookup + update	$O(n)$ simple lookup	The product is found and its quantity is changed.
Record sale	Cart validation + stock update	$O(m \times n)$	Each cart item may require product and stock validation.
Low-stock report	Product scan	$O(n)$	All active products are checked against their threshold value.
Sales by product report	Grouping sale items	$O(s)$	Sale item records are grouped and totalled by product.

Pseudo-code for key operations:

SEARCH BY NAME: load products into custom linked list; compare each product title with search text; return all matching products.

SEARCH BY BARCODE: insert products into custom hash table using barcode as key; search by entered barcode; return product or no result.

RECORD SALE: check cart items; verify stock; create sale; add sale items; reduce stock; create stock movement records; save changes.

LOW-STOCK REPORT: scan active products; compare quantity with threshold; return products that need restocking.

5. Testing Approach, Limitations and Reflection

Testing was conducted on key workflows of the project. Unit tests were designed to create products, test for duplicate barcodes, update stock, search by product-name or barcode, record sales and report low stock. Automated tests are useful because they verify important business rules after changes in code and they help maintainability (Sommerville, 2016). xUnit was selected due to its popularity as an open-source testing framework for .NET applications and its integration with .NET tooling (xUnit.net, 2026).

Test Area	Example Test Case	Expected Result
Product creation	Add a product with a unique product code and barcode.	Product is saved successfully.
Duplicate barcode	Add two products using the same barcode.	The second product is rejected.
Stock update	Change quantity for an existing product.	Quantity and stock movement are updated.
Product search	Search by name and barcode.	Matching products are returned correctly.
Sale recording	Sell a product with enough stock.	Sale is saved and stock is reduced.
Low-stock report	Set quantity below low-stock threshold.	Product appears in the low-stock report.

Limitations and Future Improvements

The system satisfies the basic needs of a small supermarket management system but it is still a local desktop solution. It is designed to be used by one small shop. There is no receipt printing, barcode scanner hardware integration, customer accounts, login roles, export to PDF/Excel or advanced sales dashboards at this time. Future enhancements could include these, alongside date filtered reports, product expiry alerts, backup and restore options and multi branch stock control.

Another limitation is the reporting is deliberately simple. The existing reports meet the brief's requirements, however, in a real supermarket, it would be useful to filter date ranges, compute profits, and export monthly summary figures. All of the above mentioned additions would increase the use of the system for commercial purposes. However, these were not included within the scope of the current implementation.

Reflection

The key concern regarding the design decision was balancing the trade-off between simplicity and completeness (for example small sized supermarket requires that the solution should also provide the necessary functionalities (ease of use) for a small supermarket). While ensuring a solution with sufficient validation, persistence, associations and search capability, I used layered architecture to decouple business logic from the user interface. A good example of this is my use of custom linked list and hash tables, which demonstrated the additional required algorithmic complexity needed for the coursework.

Conclusion

The Local Supermarket Management System is a full accounting/computerisation solution that provides the complete desktop solution for managing products, suppliers, inventory, sales, search and reporting. The Local Supermarket Management System provides a very effective means of maintaining proper, and organized records of all shop activities, and provides instant access to low stock and sales history. In addition to providing an effective way of maintaining accurate manual inventory and sales records, the project demonstrated how to successfully implement SQL Server, Entity Framework Core, algorithms, custom data structures, validation, testing and an analysis of time complexity. Overall, the totality of the system met the course requirements and provides a great foundation for continuous future development.

References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.
- Date, C.J. (2003) An Introduction to Database Systems. 8th edn. Boston: Addison-Wesley.
- Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.
- Microsoft (2024) Overview of Entity Framework Core. Available at: <https://learn.microsoft.com/en-us/ef/core/> (Accessed: 25 June 2026).
- Microsoft (2025a) Windows Forms overview. Available at: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/> (Accessed: 25 June 2026).
- Microsoft (2026) SQL Server Express LocalDB. Available at: <https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb> (Accessed: 25 June 2026).
- Sommerville, I. (2016) Software Engineering. 10th edn. Boston: Pearson.
- xUnit.net (2026) Getting Started with xUnit.net v3. Available at: <https://xunit.net/docs/getting-started/netcore/cmdline> (Accessed: 25 June 2026).

Appendix A: Key Code Snippets

The following snippets show selected parts of the implementation. They are included as supporting evidence only and are outside the main report body.

A.1 Database Context

```
public class SupermarketDbContext : DbContext
{
    public const string DefaultConnectionString =

    @"Server=(localdb)\MSSQLLocalDB;Database=LocalSupermarketDB;Trusted_Connection=True;TrustS
erverCertificate=True;";

    public DbSet<Product> Products => Set<Product>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Supplier> Suppliers => Set<Supplier>();
    public DbSet<StockRecord> StockRecords => Set<StockRecord>();
    public DbSet<Sale> Sales => Set<Sale>();
    public DbSet<SaleItem> SaleItems => Set<SaleItem>();
}
```

A.2 Search Methods

```
public List<Product> SearchByNameLinear(string searchText)
{
    var list = BuildProductLinkedList();
    return list.FindAll(p => p.Title.Contains(searchText.Trim(),
        StringComparison.OrdinalIgnoreCase)).ToList();
}

public Product? SearchByBarcodeHash(string barcode)
{
    var table = BuildBarcodeHashTable();
    return table.TryGetValue(barcode.Trim(), out var product) ? product : null;
}
```

A.3 Sale Recording

```
foreach (var item in items)
{
    var product = products.First(p => p.Id == item.ProductId);
    sale.SaleItems.Add(new SaleItem
    {
        ProductId = product.Id,
        Quantity = item.Quantity,
        UnitPrice = product.Price,
        LineTotal = product.Price * item.Quantity
    });
    product.QuantityInStock -= item.Quantity;
}
```